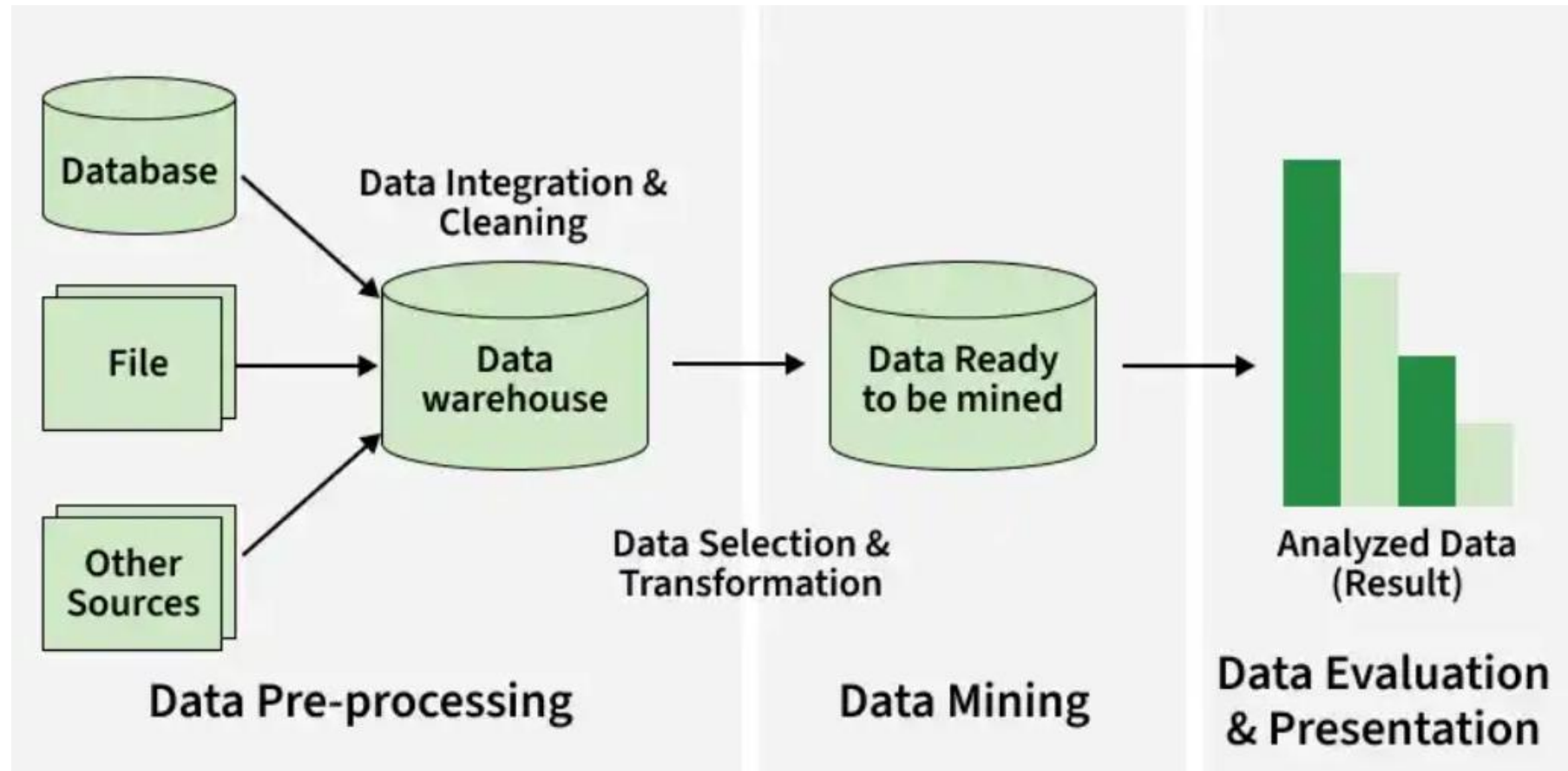


Time-Series

Lecture 12

Time series

- Data mining is the science and practice of discovering meaningful patterns, trends and knowledge from large and complex datasets.
- It integrates techniques from statistics, machine learning, databases and pattern recognition to support data-driven decision making.
- Rapid growth of digital data has made efficient and accurate data mining essential.
- Data mining is widely used in domains such as finance, healthcare, biology and web analytics.
- Sequence data is a key data type where the order of events is significant.
- Sequence data analysis reveals temporal, logical and structural patterns not visible in unordered data.



- *A sequence is an ordered list of events or elements.*
- *The order reflects time, position or logical dependency between elements*

Time-Series Data

- Time-series data consists of numerical values recorded at regular time intervals. Each observation is associated with a specific timestamp, making time an intrinsic component of the data.
- Numeric and continuous in nature
- Observations are time-dependent
- Often exhibit trends, seasonality and cyclic patterns

Time-Series Analysis and Forecasting

- Time-series analysis focuses on identifying underlying patterns such as:
- **Trend:** Long-term increase or decrease in data
- **Seasonality:** Repeating patterns over fixed periods
- **Cyclic movements:** Fluctuations over irregular periods

- Time-series forecasting uses historical data to predict future values. Common models include:
- ARIMA (AutoRegressive Integrated Moving Average)
- SARIMA (Seasonal ARIMA)
- Long-memory and advanced machine learning models
- Subsequence matching and piecewise approximation techniques are widely used to efficiently analyze large time-series databases.

Applications

- Stock market and financial analysis
- Weather and climate monitoring
- Medical signal analysis (ECG, EEG)
- Sensor and IoT data analysis

Symbolic Data

- Symbolic sequence data consists of ordered events or symbols, recorded with or without explicit time information. Unlike time-series data values are categorical rather than numeric.
- Customer purchase sequences
- Web clickstream data
- User navigation paths

Sequential Pattern Mining

- Symbolic data is mainly analyzed using sequential pattern mining which aims to discover frequent subsequences occurring across multiple sequences. The techniques used include:
- Constraint-based pattern matching
- Apriori-based algorithms
- Generalized Sequential Pattern (GSP) algorithm

- These techniques help identify user behavior patterns, product associations and navigation trends, supporting applications such as recommendation systems and marketing analysis.
- Below is an example of a symbolic data where we see customers c1 and c2 are purchasing products at different time intervals

Tid	Time	Cid	Event(purchase product)
t1	11:45:30	c1	wheat, rice, fruit
t2	11:36:50	c2	rice, fruit
t1	12:00:01	c1	juice, rice
t2	01:00:34	c2	sugar, milk

Biological Data

- Biological sequence data consists of DNA, RNA and protein sequences. These sequences are composed of nucleotides or amino acids arranged in a specific order, carrying essential biological information.
- Extremely long and complex sequences
- High dimensionality
- Contain hidden functional and structural patterns

Techniques

- Sequence alignment and indexing
- Substitution matrices and trees
- Probabilistic models
- BLAST (Basic Local Alignment Search Tool) is one of the most widely used tools for biological sequence analysis, enabling fast and efficient similarity searches in large biological database

Applications

- Gene identification and annotation
- Protein structure and function analysis
- Disease diagnosis and drug discovery

Advantages of Sequence Data Mining

- Reveals temporal and structural patterns
- Supports prediction and decision making
- Enables discoveries in finance, healthcare and bioinformatics
- Handles complex, ordered and high-dimensional data.

Basic DateTime Operations in Python

- Python has an in-built module named DateTime to deal with dates and times in numerous ways. In this article, we are going to see basic DateTime operations in Python.

- There are six main object classes with their respective components in the datetime module mentioned below:
- datetime.date
- datetime.time
- datetime.datetime
- datetime.tzinfo
- datetime.timedelta
- datetime.timezone
- Now we will see the program for each of the functions under datetime module mentioned above.

datetime.date()

We can generate date objects from the date class. A date object represents a date having a year, month and day.

Syntax

```
datetime.date( year, month, day)
```

strftime to print day, month and year in various formats. Here are some of them are:

- **current.strftime("%m/%d/%y")** that prints in month(Numeric)/date/year format
- **current.strftime("%b-%d-%Y")** that prints in month(abbreviation)-date-year format
- **current.strftime("%d/%m/%Y")** that prints in date/month/year format
- **current.strftime("%B %d, %Y")** that prints in month(words) date, year format

datetime.time()

A time object generated from the time class represents the local time.

Components:

- hour
- minute
- second
- microsecond
- tzinfo

Syntax

```
datetime.time(hour, minute, second, microsecond)
```

datetime.datetime()

datetime.datetime() module shows the combination of a date and a time.

Components:

- year
- month
- day
- hour
- minute
- second,
- microsecond
- tzinfo

Syntax

```
datetime.datetime( year, month, day )
```

```
datetime.datetime(year, month, day, hour, minute, second, microsecond)
```

Current date and time using the `strftime()` method in different ways:

- `strftime("%d")` gives current day
- `strftime("%m")` gives current month
- `strftime("%Y")` gives current year
- `strftime("%H:%M:%S")` gives current time in an hour, minute and second format
- `strftime("%m/%d/%Y, %H:%M:%S")` gives date and time together

datetime.timedelta()

- It shows a duration that expresses the difference between two date, time or datetime instances to microsecond resolution.
- Here we implemented some basic functions and printed past and future days.
- Also, we will print some other attributes of timedelta max, min that show maximum days and time, minimum date and time
- Here we will also apply some arithmetic operations on two different dates and times.

Different Functions with description under time module

Function	Description
time()	Returns the time in floating point number in seconds
ctime()	Returns the current date and time
sleep()	Stops execution of a thread for the given duration
localtime()	Returns the date and time in time.struct_time format
gmtime()	Returns time.struct_time in UTC format
mktime()	Returns the seconds passed since epochs are output
asctime()	Returns a string representing the same

1. time() method

The time() method returns the time as a floating-point number expressed in seconds since the epoch, in UTC

Syntax

```
time.time([ ])
```

Note: It does not have any parameter

```
import time  
print(time.time())
```

Output

```
1616692391.3081982
```

2. ctime() method

ctime() method converts a time expressed in seconds since the epoch to a string representing local time. The current time as returned by time() is used If secs is not provided or None. This method is equivalent to asctime(localtime(secs)). Locale information is not used by ctime() method.

Syntax

```
time.ctime([ sec ])
```

Where sec passed as an argument is the number of seconds to be converted Into string representation.

3. sleep() method

Python time method sleep() stops execution for the given number of seconds. The floating-point the number can be passed as an argument to get more precise sleep time.

Syntax

```
time.sleep([ sec ])
```

where sec passed as an argument is the number of seconds for which the process is to be stopped.

4. localtime() method

localtime() method converts number of seconds to local time. If secs is not provided or None, the current time as returned by time() is used. The dst flag is set to 1 when DST applies to the given time.

Syntax

```
time.localtime([ sec ])
```

Where sec passed as an argument is the number of seconds to be converted into struct_time representation.

5. gmtime() method

gmtime() method converts a time expressed in seconds since the Epoch to a struct_time in UTC in which the dst flag is always zero. If secs is not provided or None, the current time as returned by time() is used.

Syntax

```
time.gmtime([ sec ])
```

Where sec passed as an argument is the number of seconds to be converted into structure struct_time representation.

6. mktime() method

It is the inverse function of localtime() method. It takes an argument as struct_time or full 9-tuple and it returns a floating-point number. If the input value is not represented as a valid time, then either OverflowError or ValueError is raised.

Syntax

```
time.mktime([t])
```

Where 't' passed as an argument is a time.struct_time object or a tuple containing 9 elements corresponding to time.struct_time object

7. asctime() method

Python time method asctime() converts a struct_time representing a time as returned by gmtime() or localtime() to a 24-character string of the following form: 'Tue Mar 23 23:21:05 2021'.

Syntax

```
time.asctime([t])
```

Where 't' passed as an argument is a tuple of 9 elements or struct_time representing a time as returned by gmtime() or localtime() function.

Time Series Analysis & Visualization in Python

- Time series data is information collected in sequence over time. It shows how things change at different points, like stock prices every day or temperature every hour.
- It is used in industries such as finance, pharmaceuticals, social media, and research.
- Analyzing and visualizing this data helps us find trends, seasonal patterns, and behaviors.
- These insights support forecasting and guide better decision-making.
- The main goal is to study data in time order to extract meaningful patterns and predictions.

Concepts in Time Series Analysis

- **Trend:** It represents the general direction in which a time series is moving over an extended period. It checks whether the values are increasing, decreasing or staying relatively constant.
- **Seasonality:** Seasonality refers to repetitive patterns or cycles that occur at regular intervals within a time series corresponding to specific time units like days, weeks, months or seasons.
- **Moving average:** It is used to smooth out short-term fluctuations and highlight longer-term trends or patterns in the data.

- **Noise:** It represents the irregular and unpredictable components in a time series that do not follow a pattern.
- **Differencing:** It is used to make the difference in values of a specified interval. By default it's 1 but we can specify different values for plots.
- **Stationarity:** A stationary time series is statistical properties such as mean, variance and autocorrelation remain constant over time.
- **Order:** The order of differencing refers to the number of times the time series data needs to be differenced to achieve stationarity.
- **Autocorrelation:** Autocorrelation is a statistical method used in time series analysis to quantify the degree of similarity between a time series and a lagged version of itself.
- **Resampling:** Resampling is a technique in time series analysis that is used for changing the frequency of the data observations.

- **Types of Time Series Data**

- Time series data can be classified into two sections:
- **Continuous Time Series:** Data recorded at regular intervals with a continuous range of values like temperature, stock prices, Sensor Data, etc.
- **Discrete Time Series:** Data with distinct values or categories recorded at specific time points like counts of events, categorical statuses, etc.

- **Visualization Approaches**

- Use line plots or area charts for continuous data to highlight trends and fluctuations.
- Use bar charts or histograms for discrete data to show frequency or distribution across categories.

Thank You